CORELINK — PLANO DE INCREMENTO FUNCIONAL • NATIVIDADE-TO

PREVENÇÃO DE REQUISIÇÕES DUPLICADAS (ISLOADING & LOCK)

HJ Infor — Controle de Estado Reativo, Trava Síncrona (useRef) e Idempotência no Checkout B2B e Painel Admin

MÓDULO DE INTERFACE	PRIORIDADE	STACK	DATA DE EXECUÇÃO
Checkout B2B & Admin (UC04)	ALTA (Integridade & UX)	React 19, Next.js, Prisma ORM	18/10/2026

1. PENDÊNCIA SELECIONADA E PRIORIZAÇÃO

PRIORIDADE: ALTA

Funcionalidade Escolhida: Prevenção de Requisições Duplicadas — Adição de controle de estado (`isLoading`) nos botões de checkout e no formulário de status do Painel Admin, eliminando o envio repetido por duplo clique.

Justificativa Técnica: Em fluxos críticos de e-commerce B2B, o acionamento repetido (*double-click*) de botões de submissão pode disparar requisições concorrentes antes que o ciclo assíncrono de renderização do React aplique o atributo `disabled` no DOM. Sem bloqueio imediato e idempotência no servidor, um duplo clique no checkout ou no Painel Admin geraria cobranças/pedidos duplicados no PostgreSQL, consumo dobrado de cupons, baixa múltipla de estoque e reenvio de e-mails transacionais.

2. RESULTADO ESPERADO E COMPORTAMENTO ALCANÇADO

0% DE DUPLICIDADE

Foi implementada uma defesa em **3 camadas complementares**: (1) **Trava Síncrona Imediata (`useRef`)** nos componentes `CheckoutForm.tsx`, `carrinho/page.tsx` e `OrderStatusControl.tsx` que intercepta cliques em menos de 1ms no mesmo *tick* do JavaScript; (2) **Controle de Estado Reativo (`isLoading`)** que desabilita botões e campos (`disabled`, `aria-busy`, `pointer-events-none`) exibindo spinner animado (`Loader2`); e (3) **Guarda de Idempotência Server-Side (`order.ts`)** com janela de 15s em `createOrder` e verificação de igualdade de estado em `updateOrderStatus`.

3. PLANO DE EXECUÇÃO DETALHADO

ATIVIDADE	RESPONSÁVEL	TECNOLOGIAS	PERÍODO	STATUS
Controle de Estado `isLoading` no Checkout. Implementação de `isLoading`, `isCouponLoading` e `checkoutLockRef` em `CheckoutForm.tsx` e `carrinho/page.tsx`.	Marques Ramirez	React 19 (`useState`, `useRef`), Tailwind CSS	16/10 a 17/10/2026	✓ Concluída
Bloqueio no Formulário de Status Admin. Conversão de `OrderStatusControl.tsx` para `<form onSubmit>` com `isLoading`, `submitLockRef` e trava `!hasChanged`.	Márcio Medrado	React 19, `useTransition`, Sonner Toast, Lucide	17/10 a 18/10/2026	✓ Concluída
Guarda de Idempotência Server-Side (`order.ts`). Deduplicação de 15s em `createOrder` e bloqueio de re-execução para status idêntico em `updateOrderStatus`.	Marques Ramirez & Márcio Medrado	Next.js Server Actions, Prisma ORM, PostgreSQL	18/10/2026	✓ Concluída
Teste de Stress de Cliques Simultâneos. Execução do script `scripts/test-duplicate-prevention.mjs` simulando rajada de cliques concorrentes (< 5ms).	Taís Fernandes, Romildo & Vandelice	Node.js, Promises Concorrentes, PostgreSQL	18/10/2026	✓ Concluída
Documentação Técnica e Relatório de Evidência. Consolidação dos trechos de código, logs de teste e geração do PDF A4 de 2 páginas.	Andréa Ribeiro	Git, GitHub, Markdown, PDF Engine	18/10/2026	✓ Concluída

4. ESPECIFICAÇÃO DOS MECANISMOS DE BLOQUEIO POR COMPONENTE

DEFESA EM 3 CAMADAS

ARQUIVO / COMPONENTE	ESTADOS & TRAVAS APLICADAS	PAPEL NA ELIMINAÇÃO DO ENVIO REPETIDO POR DUPLO CLIQUE
`CheckoutForm.tsx` (Checkout de Software)	`isLoading` + `checkoutLockRef` `isCouponLoading` + `couponLockRef`	Desabilita imediatamente o botão *"Confirmar Pedido"*, o campo de cupom e o checkbox LGPD; exibe spinner `Loader2` (*"Processando Pedido..."*) e selo de bloqueio.
`OrderStatusControl.tsx` (Painel Admin — Pedidos)	`isLoading` + `isPending` `submitLockRef` + `hasChanged`	Envolve o controle em `<form onSubmit>`; desabilita `<select>` e botão *"Salvar"* durante o envio ou quando o status selecionado já está salvo no PostgreSQL.
`carrinho/page.tsx` (Carrinho B2B)	`isLoading` + `checkoutNavLockRef`	Substitui link estático por botão controlado; bloqueia cliques múltiplos e exclusão de itens enquanto redireciona (*"Redirecionando ao Checkout..."*).
`src/actions/order.ts` (Server Actions)	`duplicateWindow (15s)` `previousStatus === newStatus`	Garante idempotência no PostgreSQL: se duas requisições chegarem juntas, retorna o pedido existente (`deduplicated: true`) sem duplicar registro ou estoque.

5. EVIDÊNCIA VISUAL DE INTERFACE E BLOQUEIO DE CLIQUES CONCORRENTES

VALIDAÇÃO HOMOLOGADA

CLIQUES SIMULTÂNEOS (TESTE)	REQUISIÇÕES DISPARADAS	CLIQUES DUPLICADOS RETIDOS	JANELA DE IDEMPOTÊNCIA DB
3 Cliques (< 5ms)	1 Única Action	2 Bloqueados (100%)	15.000 ms (15s)

1. Botão de Checkout B2B (CheckoutForm.tsx)

Transição instantânea de estado ao clicar em "Confirmar Pedido"

Confirmar Pedido (Estado Inicial — Habilitado)

Processando Pedido... (disabled=true | aria-busy=true)

Requisição em andamento — envio bloqueado contra duplo clique

2. Formulário de Status no Painel Admin (OrderStatusControl.tsx)

Bloqueio simultâneo do <select> e do botão submit durante persistência

Processando (disabled) | Salvando... (disabled)

✓ Concluído (Completed) | ✓ Salvo!

Bloqueado contra duplo clique • Desabilitado quando !hasChanged

6. IMPLEMENTAÇÃO TÉCNICA (CHECKOUTFORM.TSX / ORDERSTATUSCONTROL.TSX) E LOG DE TESTE

REACT 19 + PRISMA ORM

6.1 Controle isLoading + Trava useRef (Checkout & Admin)

```
// CheckoutForm.tsx — Bloqueio de Duplo Clique
const [isLoading, setIsLoading] = useState(false);
const checkoutLockRef = useRef(false);

const handleCheckout = async (e?: React.FormEvent) => {
  if (e) e.preventDefault();
  if (checkoutLockRef.current || isLoading || !lgpdConsent) return;

  checkoutLockRef.current = true;
  setIsLoading(true);
  const res = await createOrder(product.id, appliedCoupon?.id);
  // ...
};

// OrderStatusControl.tsx — Formulário Admin
const [isLoading, setIsLoading] = useState(false);
const [isPending, startTransition] = useTransition();
const submitLockRef = useRef(false);
const busy = isLoading || isPending;

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (submitLockRef.current || busy || !hasChanged) return;
  submitLockRef.current = true;
  setIsLoading(true);
  // ...
};
```

6.2 Guarda Server-Side (order.ts) & Log de Concorrência

```
// src/actions/order.ts — Janela de Idempotência (15s)
const duplicateWindow = new Date(Date.now() - 15_000);
const existingRecentOrder = await prisma.order.findFirst({
  where: {
    userId: session.user.id, productId: product.id,
    status: "PENDING", createdAt: { gte: duplicateWindow },
  },
});
if (existingRecentOrder) {
  return { success: true, orderId: existingRecentOrder.id, deduplicated: true };
}

=====================================================================
 TESTE DE PREVENCAO DE REQUISICOES DUPLICADAS (ISLOADING + LOCK)
=====================================================================
 [ACEITO]    Clique #1 iniciou requisicao (isLoading=true, lockRef=true)
 [BLOQUEADO] Clique #2 rejeitado (isLoading=true, lockRef=true)
 [BLOQUEADO] Clique #3 rejeitado (isLoading=true, lockRef=true)
---------------------------------------------------------------------
 Resultado UI: 1 requisicao enviada | 2 cliques duplicados bloqueados
 Ultimo pedido verificado no PostgreSQL: #CMVE00WT (COMPLETED)
=====================================================================
```

7. ROTEIRO DE VALIDAÇÃO E TESTES DE HOMOLOGAÇÃO

CENÁRIO DE TESTE	PROCEDIMENTO EXECUTADO	RESULTADO OBTIDO NA HOMOLOGAÇÃO
Duplo Clique no Botão "Confirmar Pedido"	Acionar rapidamente 3 cliques consecutivos no botão de submissão do `CheckoutForm.tsx`.	O 1º clique ativou `checkoutLockRef` e `isLoading=true` (spinner *"Processando Pedido..."*); os cliques #2 e #3 foram descartados com **1 único pedido gravado**.
Duplo Clique no Formulário de Status Admin	Selecionar *"Completed"* em `/admin/pedidos` e clicar repetidamente no botão *"Salvar"*.	O `<select>` e o botão entraram em estado `disabled` (*"Salvando..."* + selo de bloqueio), executando apenas 1 transação no Prisma ORM.
Tentativa de Salvar Status Sem Alteração	Manter o mesmo status já persistido no banco e verificar disponibilidade do botão *"Salvar"*.	Como `hasChanged === false` (`status === persistedStatus`), o botão permanece desabilitado, evitando queries `UPDATE` desnecessárias.
Idempotência Server-Side (`order.ts`)	Disparar duas chamadas concorrentes para `createOrder` e `updateOrderStatus` na mesma janela de 15s.	A Server Action identificou o registro recém-criado/atualizado e retornou `deduplicated: true` sem duplicar pedido, estoque ou e-mail.